

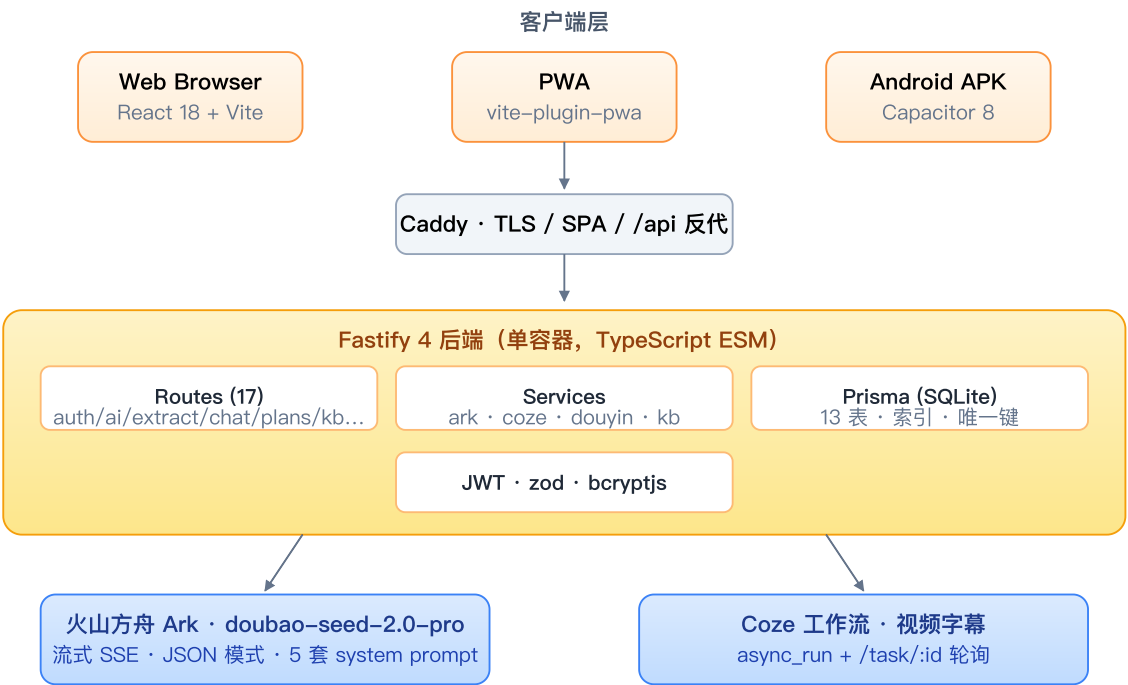
学海小书院 · 技术文档

AI 驱动的「视频 → 课程 → 陪学」学习伴侣。粘贴一条学习视频链接或抖音分享口令，30-120s 内完成：字幕提取 → 结构化大纲 → 知识库切块 → 流式陪学问答 → 学习计划编排。全链路真接 LLM，无 mock。

1. 功能版图

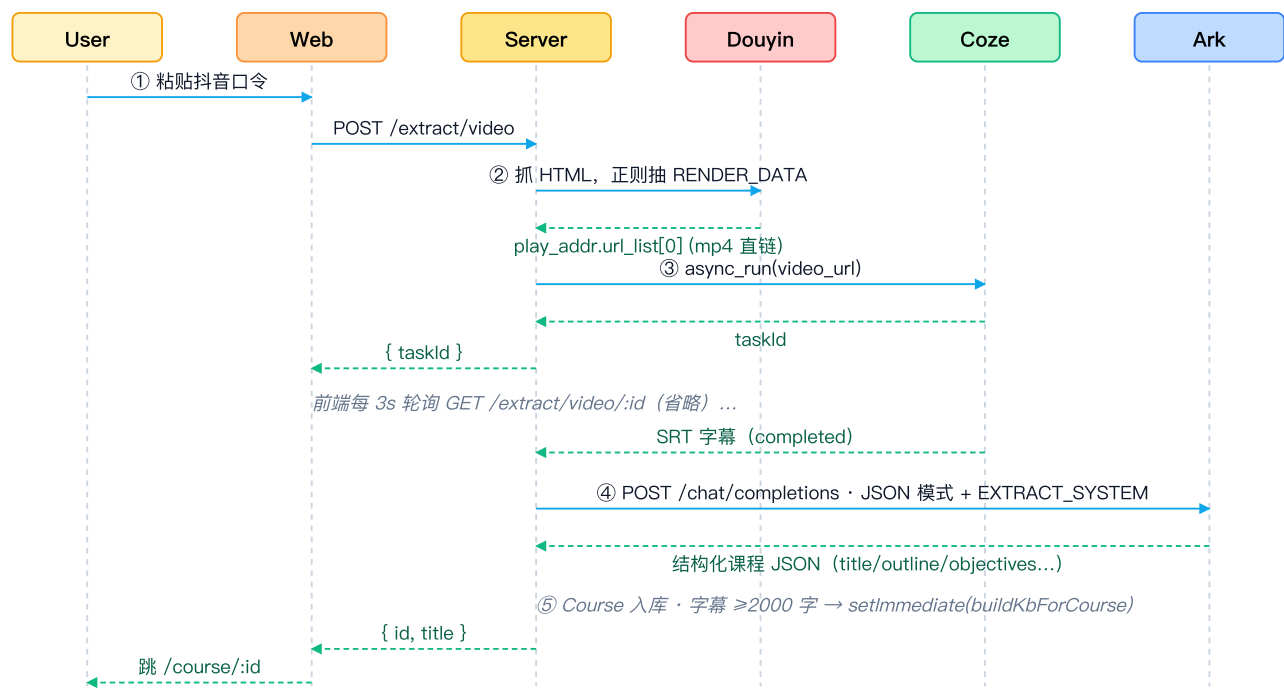
模块	能力
视频解析	链接/分享口令 → 字幕 → AI 结构化大纲（标题/章节/学习目标/前置/Tips/资源）
抖音兼容	HTML 分享页四层兜底 → mp4 直链
视频搜索	关键词检索候选 → 多选批量解析（AI 可自主触发）
AI 陪学	流式聊天 + 5 类指令标记（番茄钟/计划草稿/任务掌握/复习/视频搜索）
知识库 RAG	字幕静默切块 + 关键词检索增强
学习计划	AI 一键排期 + 真实打卡 + 番茄钟联动
跨端	Web + PWA + Android APK（Capacitor）

2. 技术架构



技术栈：React 18 + TS + Tailwind + Zustand + TanStack Query + Framer Motion · Fastify 4 + Prisma 5 + SQLite + JWT + zod · Capacitor 8 · Docker + Caddy。

3. 核心数据流：粘贴链接 → 课程大纲



AI 陪学的流程类似，区别在于：`/ai/chat` 是 SSE 流式响应；服务端先 `searchMultiKb()` 拿到 top-3 字幕片段塞进 system prompt，让模型基于片段作答。

4. AI 模型与能力

提供方	模型 / 工作流	调用	场景
火山方舟	doubao-seed-2.0-pro	OpenAI 兼容 <code>/chat/completions</code> ：SSE 流式 + <code>response_format: json_object</code>	大纲生成、陪学、计划生成、知识库切块、关键词提取
Coze	自建「视频字幕」工作流	<code>async_run</code> + 轮询 <code>/task/:id</code>	视频 URL → SRT 字幕

Ark 客户端 3 个出口（`services/ark.ts`）

- `arkChat()` — 非流式
- `arkChatStream()` — `AsyncGenerator` 逐 delta yield，前端 SSE 转 fetch ReadableStream
- `arkJson<T>()` — JSON 模式 + 失败把上次错误回喂模型自我纠错：

```
for (let i = 0; i < 2; i++) {
  const out = await arkChat({ ...opts, messages, responseFormat: 'json_object' })
  try { return JSON.parse(out.trim().replace(/^```json|```$/g, '').trim()) as T }
  catch {
    messages.push({ role: 'assistant', content: out })
    messages.push({ role: 'user', content: '上一次回复不是合法 JSON，请只输出 JSON。' })
  }
}
```

标记协议（替代 function-calling）

让模型把工具调用以**内联文本标记**形式发出，前端边流边识别边渲染对应组件。比 OpenAI tool-use 在中文场景更稳，且不必等整条消息完成就能给用户即时反馈。

标记	触发	前端渲染
<<TIMER:25:专注>>	用户开始任务	PomoTimer 自动开始
<<PLAN:goal=... weeks=4 hours=6 courseIds=a,b>>	用户要排计划	PlanProposalCard 一键创建
<<TASK_DONE:id>>	考核 ≥80% 正确率	"已掌握" chip
<<TASK_REVIEW:title=... date=...>>	考核 30–79%	"已加复习任务"
<<VIDEO_SEARCH:关键词 reason=...>>	KB 未覆盖	琥珀卡，点开打开搜索 Sheet 批量解析

system prompt 同时给出**完整示例输出**提升遵循率：

【场景 4】用户问到 KB 没覆盖的知识点：

1) 先承认 "资料里没细讲~" 2) 给 ≤80 字兜底解释 3) 末尾必须输出 <<VIDEO_SEARCH:...>>

示例：资料里没有这部分呢~简单说，闭包是函数记住了它定义时的外层变量.....

<<VIDEO_SEARCH:JavaScript 闭包 动画讲解|reason=动画 + 实战例子比纯文字好懂>>

轻量级 RAG（无向量库）

中文学习场景下关键词足够。构建时让 doubao 同时**切块 + 标 5–12 个关键词**写库；检索时再让 doubao 把问句拆 3–10 个关键词，按"关键词命中 ×3 + 文本命中 ×1"打分取 top-3：

```
const scored = rows.map(r => {
  let score = 0
  for (const w of kw) {
    if (r.keywords.includes(w)) score += 3
    if (r.text.includes(w)) score += 1
  }
  return { ord: r.ord, text: r.text, score }
})
return scored.filter(x => x.score > 0).sort((a,b)=>b.score-a.score).slice(0, k)
```

5. 核心实现要点

5.1 抖音 HTML → mp4（ services/douyin.ts ）

四层兜底，单层失败自动落到下一层：

- 1. 用 iPhone UA 跟随短链 302 到 iesdouyin.com/share/video/<id>/
- 2. 解析 <script id="RENDER_DATA">（URI 编码 JSON）→ 递归找 play_addr.url_list[0]
- 3. 解析 window._ROUTER_DATA = {...}
- 4. 抠 item_ids 调老 iteminfo 接口
- 5. 最后用 Range: bytes=0–1 极小请求触发 302 拿 CDN final URL，并校验 content-type 含 video/

实测一条真实分享口令端到端 ~1.5s 拿到 720p mp4 直链。

5.2 全局后台任务（ store/bgTasks.ts ）

视频解析 30–120s，不能阻塞页面。Zustand store 维护 BgTask[]，每个任务有 stage / progress / stageLabel；任务 Promise 持有在 store，**用户切走页面继续跑**；BackgroundTaskFloater 全局挂在 App.tsx，右下角圆形按钮 + 抽屉跨页可见；完成时走 @capacitor/local-notifications 本地通知。

批量解析用 worker 池 concurrency=3 防 Coze QPS 超限：先一次性把 N 个 BgTask 注册显示「🕒 排队等待中...」，再受控出队真正调用：

```
const queue = items.map(it => { const id = nanoid(); store.add({ id, stageLabel: '🕒 排队等待中...', ... }); return id });
let active = 0, cursor = 0
const tick = () => {
  while (active < concurrency && cursor < queue.length) {
    const job = queue[cursor++]
    if (store.tasks.find(x => x.id === job.id)?.stage === 'cancelled') continue
    active++
    runVideoExtract(job.id, job.input).finally(() => { active--; tick() })
  }
}
```

5.3 流式 SSE 解析 (services/ark.ts)

注意半行 buffer，避免跨 chunk 的 SSE 行被切断：

```
let buf = ''
while (true) {
  const { value, done } = await reader.read(); if (done) break
  buf += decoder.decode(value, { stream: true })
  const lines = buf.split('\n'); buf = lines.pop() ?? '' // 半行留到下次
  for (const raw of lines) {
    const line = raw.trim(); if (!line.startsWith('data:')) continue
    const payload = line.slice(5).trim(); if (payload === '[DONE]') return
    const delta = JSON.parse(payload)?.choices?.[0]?.delta?.content
    if (delta) yield delta
  }
}
```

5.4 标记解析 (pages/Chat.tsx)

边流边 extractAllTags()，识别后内联渲染对应组件并从纯文本里剥离：

```
for (const m of content.matchAll(/<<VIDEO_SEARCH:([^\>]+)>>/g)) {
  const parts = m[1].trim().split('|').map(s => s.trim())
  const head = parts[0]
  const kv: Record<string, string> = {}
  for (const p of parts.slice(1)) { const i = p.indexOf('='); if (i > 0) kv[p.slice(0, i).trim()] = p.slice(i + 1).trim() }
  const keyword = !head.includes('=') ? head : (kv.keyword || kv.q || '')
  if (keyword) tags.push({ kind: 'video_search', key: `vs-${idx++}`, keyword, reason: kv.reason })
}
```

5.5 解析完成自动加入对话引用

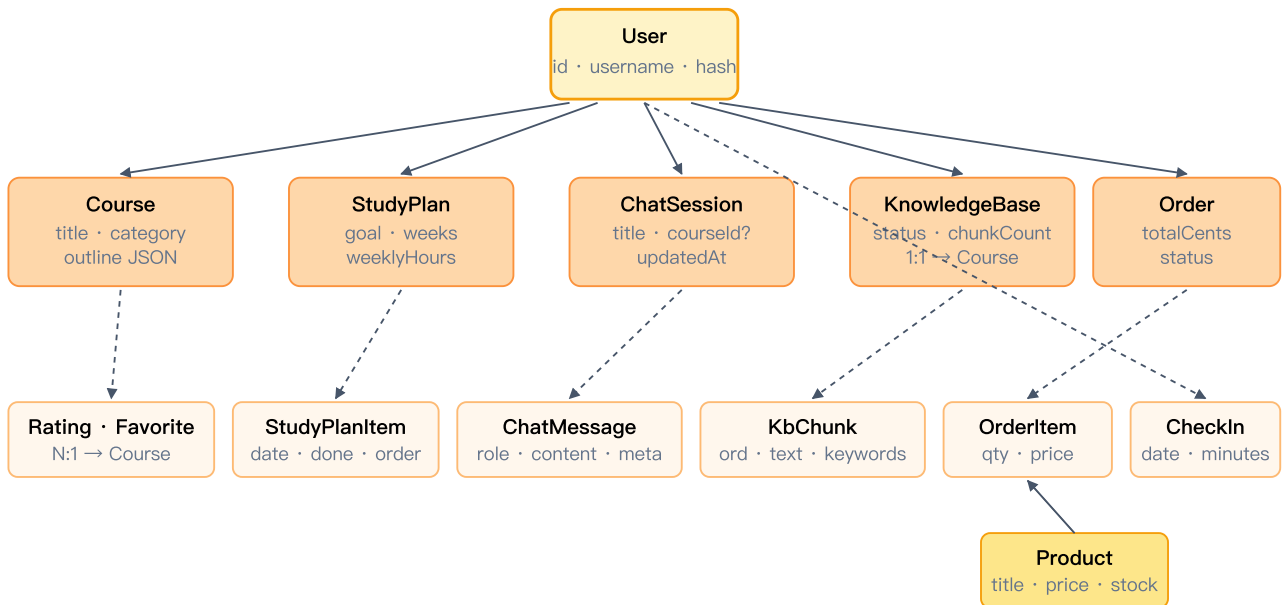
Chat 订阅 bgTasks store，监控自己提交的 taskIds，完成时把新 courseId 加进 citedCourseIds，AI 下一轮回答就能基于新素材：

```
useEffect(() => useBgTasks.subscribe((state) => {
  for (const id of pendingRef.current) {
    const t = state.tasks.find(x => x.id === id)
    if (t?.stage === 'done' && t.result?.id) {
      setCitedCourseIds(prev => prev.includes(t.result.id) ? prev : [...prev, t.result.id])
      toast.success(`已自动引用《${t.result.title.slice(0, 18)}》到当前对话`)
      pendingRef.current.delete(id)
    }
  }
}), [])
```

5.6 剪贴板感知 (hooks/useClipboardLink.ts)

mount 延迟 600ms (避开 splash) + `visibilitychange→visible` + `window focus` 三个时机读 `navigator.clipboard` ; 内存 `lastSeenRef` 防重弹 + `localStorage` 持久化忽略列表; APK 上 Capacitor WebView 原生支持, 无需额外插件。

6. 数据模型



实线=直接外键 · 虚线=1:N 子表 · 共 13 张表 (其余略)

要点: JSON 字段 (tags / outline / objectives) 以 TEXT 存 JSON 字符串, 路由层统一 `serialize()` 反序列化;
`CheckIn(userId,date,courseId)` 唯一键防重复打卡; `KnowledgeBase.courseId ON DELETE SET NULL` 删课程不丢知识库; `KbChunk` / `ChatMessage` / `StudyPlanItem` 级联删除。

7. 部署与运行

```
npm install && npm run db:push -w server && npm run db:seed -w server
npm run dev # 本地: server 8787 + web 5173
docker compose up -d --build # 生产: Dockerfile 3 阶段构建 + Caddy 自动 TLS
```

关键环境变量: `API_KEY` (火山方舟) · `ARK_MODEL` (默认 `doubao-seed-2.0-pro`) · `JWT_SECRET` · `COZE_BASE_URL` / `COZE_API_TOKEN` · `DATABASE_URL` (默认 `SQLite`) 。

关键文件索引: `server/src/services/{ark,coze,douyin,kb}.ts` · `server/src/routes/{ai,extract,chat,plans}.ts` · `web/src/store/bgTasks.ts` · `web/src/pages/{Home,Chat,Library,Extract}.tsx` · `web/src/components/{VideoSearchSheet,BackgroundTaskFloater,ClipboardLinkPrompt,PlanProposalCard}.tsx` · `web/src/hooks/useClipboardLink.ts` 。

代码量: 后端 ~2,400 LOC, 前端 ~6,300 LOC, 合计 ~9,000 行 (不含生成代码) 。